TP N.º 8 · Food Store en la feria

UTN — Tecnicatura Universitaria en Programación · Base de Datos II · Ciclo 2026

Equipo: Ponce De Leon Leonel · Storani Franco · Pinto Guillermo · Soto Matias

Alcance: desarrollamos una simulación SQLite reproducible y documentamos su integración. Revisamos los archivos adjuntos de TP7; no pudimos recuperar todos los scripts anteriores del repositorio. No ejecutamos nuestro PostgreSQL original ni Kiro/OpenCode Plan y registramos esas validaciones como pendientes.

1.a · Qué datos viajan a la tablet

Dato	¿Viaja?	Motivo / autoridad
Producto: id, nombre	Sí	Identificar/mostrar el catálogo offline. Autoridad PostgreSQL.
Precio en centavos, stock visto	Sí (copia)	Mostrar valores observados, nunca autorizar venta con ellos.
Disponible, eliminado, fecha de copia	Sí	Omitir bajas del catálogo visible; guardar lápidas.
Contraseñas, roles, email, datos bancarios	No	No hacen falta en tablet; minimiza exposición.
Total de ventas y pedido confirmado	No	Se crean solo en PostgreSQL después de validación real.

La tablet es una copia temporal y una bandeja de borradores. La fuente de verdad de precio, stock, baja lógica y ventas confirmadas es PostgreSQL; MongoDB recibe documentos para lecturas históricas, no decide el inventario.

1.b–1.c · Esquema y pruebas de base local

Implementamos cuatro tablas STRICT: producto_cache, pedido_borrador, item_borrador y cola_sync. Agregamos snapshot_servidor para importación. PRAGMA foreign_keys = ON habilita las FK. CHECK valida precio no negativo, cantidades positivas, banderas binarias, forma de pago y estados. Cada ítem referencia borrador y producto; los precios son INTEGER en centavos.

Nuestra transacción de ejemplo escribe borrador + ítem + solicitud dentro de BEGIN/COMMIT; otra inserción se revierte con ROLLBACK. En C1 consultamos localmente los seis productos activos de la muestra. En C7 provocamos dos errores reales: cantidad 0 → CHECK; producto ID 999999 → FOREIGN KEY. Ninguno agrega filas.

2 · Instantánea y semáforo

2.a · Snapshot desde PostgreSQL

Para nuestra demostración utilizamos `postgresql/01_demo_postgresql.sql` y creamos `tp8_demo.v_snapshot_movil` con id, nombre, precio_centavos, stock, disponible, eliminado y fecha_copia. En el flujo real se exporta CSV con `\copy` y se importa a `snapshot_servidor` de SQLite. Incluimos una muestra ficticia consistente con la demostración; no es una exportación comprobada de nuestro PostgreSQL original. Fecha de referencia de la muestra: 2026-11-10.

```
\copy (SELECT * FROM tp8_demo.v_snapshot_movil ORDER BY id) TO 'servidor_snapshot.csv' CSV HEADER
```

En la vista convertimos los booleanos a INTEGER 0/1 desde la vista, en lugar de importar t/f a tablas STRICT. `cargar_snapshot.sql` realiza UPSERT hacia producto_cache; las bajas lógicas se mantienen para integridad referencial, aunque desaparecen de la consulta de catálogo activo.

2.b · Reglas de decisión

Prioridad	Condición	Resultado y motivo
Rojo	No existe, no disponible, eliminado, falta stock o borrador vacío	RECHAZADO: producto inválido o stock insuficiente.
Amarillo	Sin rojo, pero precio actual difiere del visto	CONFLICTO: aceptar nueva cotización y emitir nuevo UUID.
Verde	Existe, activo, cantidad válida y precio igual	SIMULADO_APROBADO: validación local, NO venta.

Nuestra consulta actualiza únicamente `cola_sync` de SQLite donde estado = PENDIENTE. Una segunda ejecución no reevalúa estados finales. El orden rojo > amarillo evita aprobar solicitudes con faltantes de stock aunque el precio también haya cambiado.

Control de integridad y reproducibilidad

Ejecutamos las pruebas sobre SQLite 3.46.1 mediante `python tests/verificar.py`: `PRAGMA integrity_check` devolvió `ok` y `PRAGMA foreign_key_check` no devolvió violaciones. Verificamos un ROLLBACK que deja cero filas y las condiciones de C7 con excepciones de integridad.

2.c · Casos C2 a C6

Utilizamos una secuencia de instantáneas simuladas en SQLite como representación de nuevas exportaciones del servidor. Documentamos en el README cómo reproducir estos pasos en un PostgreSQL real de pruebas. En los cinco casos el semáforo solo escribe SQLite.

Caso y comandos de preparación	Resultado observado	Interpretación
C2 Borrador Muzzarella 100000; `UPDATE snapshot_servidor SET precio_centavos=110000 WHERE id=1`; semáforo; aceptar en item 110000, INSERT nuevo UUID y semáforo.	CONFLICTO → nuevo UUID SIMULADO_APROBADO. El anterior sigue CONFLICTO.	El precio visto dejó de ser válido; nueva solicitud reevalúa tras aceptar.
C3 Borrador Napolitana cantidad 9; stock de snapshot = 8; semáforo.	RECHAZADO.	No alcanzan unidades; no se reserva ni descuenta stock.
C4 Borrador Carne; `UPDATE snapshot_servidor SET disponible=0, eliminado=1 WHERE id=4`; refrescar caché y semáforo.	RECHAZADO; fila id=4 retenida pero invisible en catálogo activo.	La lápida preserva referencias históricas de borradores.
C5 Borrador Coca 1.5L, precio 80000 y cantidad 1; sin modificaciones; semáforo.	SIMULADO_APROBADO.	Cumple reglas de la instantánea, sin confirmar pedido.
C6 Duplicar INSERT en cola_sync con mismo solicitud_uuid; ejecutar semáforo otra vez.	UNIQUE constraint failed; no cambian estados anteriores.	Idempotencia local; en producción debe existir también en endpoint/API y tabla PostgreSQL, con clave única y transacción.

Control de no mutación del servidor

El script `postgresql/04_control_no_mutacion.sql` consulta cantidad de pedidos, lista de stocks y suma de stock antes y después. Nuestra demostración inicial define 2 pedidos y stock total 67; el módulo SQLite no contiene ninguna conexión ni sentencia que escriba PostgreSQL. No realizamos una comparación ejecutada en nuestro PostgreSQL original: falta correr la consulta en la base real del grupo y agregar evidencia.

3 · Capa de datos, operación y revisión

3.a · Mapa de los tres motores

Motor	Qué almacena	Fuente de verdad y límites
PostgreSQL	Productos, precios, stock, usuarios, pedidos y líneas confirmadas.	Autoritativo para operación comercial. Valida y registra venta real en transacción.
MongoDB (Unidad 7)	Documentos `pedidos` con cliente básico y `lineas` embebidas; precio histórico y estado.	Proyección de lectura histórica. Se alimenta desde datos confirmados; no decide stock ni precio actual.
SQLite (tablet, Unidad 8)	`producto_cache`, borradores, ítems, cola y snapshot.	Copia temporal offline. Puede proponer solicitudes, no confirmar ventas.

En nuestro TP7 adjunto, MongoDB utiliza pedidos con líneas embebidas y consultas por estado CANCELADO y por producto Carne. PostgreSQL conserva tablas relacionales `pedido`, `detalle_pedido` y `producto` según el SQL provisto.

3.b · Plan de mantenimiento

Respaldo y restauración: respaldar PostgreSQL con `pg_dump` y probar restauración con `pg_restore` en base aislada, verificando conteos, integridad y transacciones. Respaldar MongoDB con herramienta equivalente y verificar lectura histórica; proteger o exportar borradores SQLite pendientes antes de borrar/reinstalar tablets.

Si el respaldo de PostgreSQL es antiguo: suspender sincronización y confirmar el punto de restauración. Reconciliar solicitudes e identificadores ya procesados, publicar nueva versión/época de catálogo y forzar recarga en tablets antes de volver a aceptar operaciones. Nunca reenviar a ciegas pedidos ya confirmados.

Frecuencia y responsable: durante ferias, el operador revisa CONFLICTO/RECHAZADO al recuperar señal y en el cierre del turno; el administrador valida pendientes y excepciones diariamente. Registrar fecha, causa, responsable y acción correctiva sin cambiar ventas automáticamente.

3.c · Análisis preliminar de IA

En la página siguiente registramos propuestas aceptadas, corregidas y descartadas junto con las pruebas disponibles. Esto no reemplaza la revisión real exigida por la cátedra en OpenCode Plan sobre todos los scripts de la cursada: dicho paso requiere acceso a TP1–TP7 y a la herramienta indicada.

3.c · Hallazgos y cierre

Unidad / hallazgo	Decisión y justificación	Cómo se comprobó
U8: cantidades cero rompen el borrador.	Corregido. CHECK(cantidad > 0) en item_borrador.	C7: INSERT con 0 produjo CHECK; conteo de filas no aumentó.
U8: borrar físicamente un producto dado de baja compromete FK de borradores.	Corregido. Guardar lápida eliminado=1, ocultar de catálogo.	C4: id=4 permanece, consulta activa no lo muestra; FK sin violaciones.
U7: propuesta de normalizar los documentos históricos quitando líneas y cliente embebido.	Descartado. El TP7 justifica documentos autocontenidos para tickets, historia de precios y lectura completa.	Comprobación documental en `consultas_mongodb.js` y PDF TP7; validación en instancia MongoDB pendiente.
U8: autorizar stock desde la tablet.	Descartado. Viola “el móvil propone, el servidor dispone”.	Inspección de semaforo.sql: solo UPDATE de cola_sync; control real PostgreSQL pendiente.

4 · Kiro, bitácora, DUIA y defensa

Adjuntamos `docs/kiro/requirements.md` con cuatro requisitos EARS vinculados a C2, C3-C4, C5 y C6, además de `design.md` y `tasks.md`. También incluimos `bitacora_ia.md`, `DUIA.md`, README y guion de defensa de diez minutos. Utilizamos ChatGPT como apoyo para esta entrega y ejecutamos pruebas SQLite; no atribuimos a Kiro ni OpenCode tareas que no realizamos.

Limitaciones explícitas antes de la entrega definitiva

1) Debemos adaptar la vista y el CSV a las columnas del PostgreSQL original y probarlos sobre una copia. 2) Tenemos que completar la revisión de TP1–TP6 en OpenCode Plan. 3) Nos queda cargar la spec dentro de Kiro. 4) Debemos cotejar la DUIA con el formato oficial de la cátedra. Ya verificamos la solución local y los casos C1-C7.

Referencia: enunciado “BDII_U8_TP8_Food_Store” y TP7 Food Store adjuntados por el equipo; repositorio público github.com/fstoutn/bd2_tps (estructura visible).